Apéndice B. Manual Técnico

SISTEMA DE GESTIÓN DE ACTIVOS TECNOLÓGICOS

Manual Técnico

Documentación de instalación, configuración y mantenimiento del sistema

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Kevin Humberto Del Cid Morales

Versión 1.0 — Agosto de 2026

Introducción

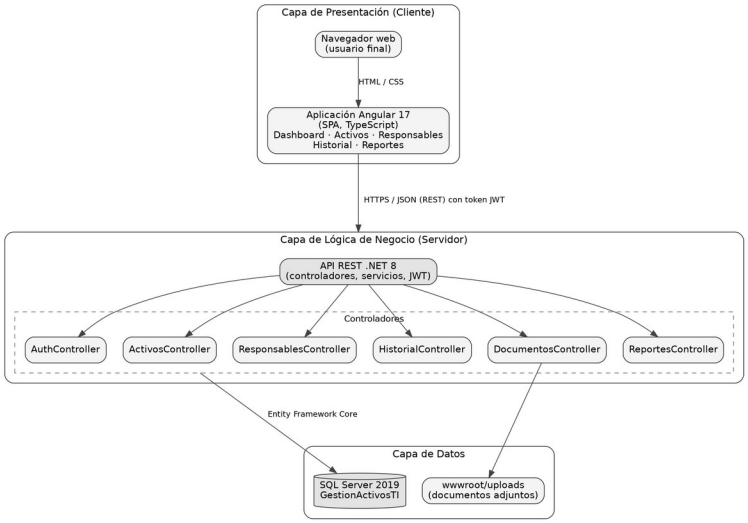
Este manual documenta la construcción interna del sistema de gestión de activos tecnológicos y está dirigido al personal de soporte, mantenimiento y desarrollo. Su objetivo es que un técnico ajeno al desarrollo original pueda instalar el sistema desde cero, configurarlo, ponerlo en producción y mantenerlo operativo.

El documento describe la arquitectura, el modelo de datos, las tecnologías empleadas, la estructura del código fuente, la configuración del entorno, el procedimiento de despliegue y las tareas de mantenimiento preventivo y correctivo. Se asume conocimiento de desarrollo web, de administración de bases de datos relacionales y del entorno de ejecución de .NET.

Arquitectura del Sistema

El sistema se construyó siguiendo el modelo cliente-servidor de tres capas. La capa de presentación es una aplicación de página única desarrollada en Angular que se ejecuta en el navegador del usuario. La capa de lógica de negocio es una interfaz de programación de aplicaciones de tipo REST desarrollada en C# sobre .NET 8. La capa de datos corresponde a una base de datos relacional administrada por Microsoft SQL Server.

Arquitectura de tres capas del sistema.



Nota. Elaboración propia.

La comunicación entre la capa de presentación y la de negocio se realiza mediante peticiones HTTP con contenido en formato JSON. Cada petición posterior al inicio de sesión incluye el token de sesión en el encabezado de autorización; un interceptor del cliente lo agrega de forma automática. La capa de negocio no expone las entidades del modelo de datos: la frontera está mediada por objetos de transferencia de datos que definen de forma explícita qué información entra y qué información sale.

Dentro de la capa de negocio se aplica una separación adicional en tres niveles. Los controladores reciben la petición, resuelven la autorización y validan el contrato de entrada. Los servicios contienen las reglas de negocio y se registran siempre detrás de una interfaz, lo que permite sustituirlos en las pruebas automatizadas. El contexto de datos, construido sobre Entity Framework Core, resuelve la persistencia.

Tabla 95

Componentes de la Capa de Lógica de Negocio

Componente	Responsabilidad
AuthService	Verificación de credenciales con BCrypt y confirmación de contraseña.
JwtService	Emisión del token de sesión y cálculo de su vigencia.
ActivosService	Alta, consulta, actualización, baja lógica, asignación de responsable y gestión de componentes tecnológicos.
ResponsablesService	Catálogo de responsables, baja lógica con traspaso de activos y reactivación.
HistorialService	Consulta paginada de la bitácora por categoría.
DocumentosService	Almacenamiento físico de los archivos adjuntos y sus metadatos.
ReportesService	Cálculo de los cuatro informes y del resumen del tablero.

Diseño de la Base de Datos

La base de datos se denomina GestionActivosTI y utiliza la intercalación SQL_Latin1_General_CP1_CI_AS, que no distingue mayúsculas ni acentos en las comparaciones de texto. Está compuesta por diez tablas, cinco vistas, cuatro procedimientos

almacenados, dos funciones escalares y tres disparadores, cuyo detalle se documenta en la sección 4.3 de este trabajo.

El modelo entidad-relación completo, con las diez tablas y sus relaciones, se presenta en la sección 4.3 de este trabajo.

La entidad Activos constituye el núcleo del modelo. La tabla ActivoResponsable resuelve la relación entre activos y responsables conservando la vigencia de cada asignación mediante las fechas de inicio y finalización y el indicador de asignación actual, lo que permite reconstruir la custodia de un activo en cualquier fecha pasada. La tabla HistorialCambios almacena la bitácora funcional escrita por la capa de servicios, mientras que AuditoriaActivos almacena el rastro técnico campo por campo, alimentado por un disparador.

El borrado de información es lógico en todos los casos de negocio: los activos pasan al estado Retirado y los responsables al estado inactivo. La eliminación física únicamente ocurre sobre componentes tecnológicos y documentos adjuntos, por tratarse de datos accesorios del activo.

Los archivos de comando de la base de datos se encuentran en la carpeta database del proyecto y deben ejecutarse en el orden que indica la tabla siguiente.

Tabla 96

Archivos de Comando de la Base de Datos

Orden	Archivo	Contenido
1	01_database.sql	Creación de la base de datos, las nueve tablas iniciales, los índices y la carga de datos base: roles, tipos de activo y usuarios de prueba.
2	02_componentes_costos.sql	Incorporación de las columnas de costo, moneda y días de aviso, y creación de la tabla de componentes tecnológicos.
3	03_objetos_bd.sql	Creación de la tabla de auditoría y de los objetos de programación: vistas, procedimientos almacenados, funciones escalares y disparadores.

Tecnologías Utilizadas

Tabla 97

Tecnologías y Versiones Empleadas en el Desarrollo

Capa	Tecnología	Versión	Función que cumple
Presentación	Angular	17	Marco de trabajo del cliente web, con componentes autónomos y la nueva sintaxis de control de flujo.
Presentación	Angular Material	17	Biblioteca de componentes de interfaz.
Presentación	Chart.js	4	Generación de las gráficas del tablero y del resumen.
Presentación	TypeScript	5.4	Lenguaje de programación del cliente.
Negocio	.NET	8.0	Plataforma de ejecución de la interfaz de programación.
Negocio	ASP.NET Core	8.0	Marco de trabajo de la API REST.
Negocio	Entity Framework Core	8.0.0	Mapeo objeto-relacional y acceso a datos.
Negocio	BCrypt.Net-Next	4.0.3	Resumen y verificación de contraseñas.
Negocio	Microsoft.AspNetCore. Authentication.JwtBearer	8.0.0	Validación del token de sesión.
Negocio	System.IdentityModel. Tokens.Jwt	7.0.3	Emisión y firma del token.
Negocio	Swashbuckle.AspNetCore	6.5.0	Documentación interactiva de la API.
Datos	Microsoft SQL Server	2019	Motor de base de datos relacional.
Pruebas	xUnit	2.x	Marco de trabajo de las pruebas automatizadas.
Pruebas	EntityFrameworkCore. InMemory	8.0.0	Base de datos en memoria para las pruebas.
Pruebas	AspNetCore.Mvc.Testing	8.0.0	Servidor en memoria para pruebas de integración.

Estructura del Proyecto

El repositorio se organiza en tres carpetas principales: database, backend y frontend. La estructura del servidor es la siguiente.

```
backend/
├── GestionActivosTI.API/
│   ├── Controllers/     Puntos de entrada HTTP
│   ├── Services/        Reglas de negocio tras interfaz
│   ├── Models/          Entidades de Entity Framework
│   ├── DTOs/            Contratos de entrada y salida
│   ├── Data/            AppDbContext
│   ├── Middleware/      Manejo global de excepciones
│   ├── wwwroot/uploads/ Documentos adjuntos
│   ├── Program.cs       Configuración y arranque
│   └── appsettings.json Conexión, JWT y CORS
└── GestionActivosTI.API.Tests/
    ├── Services/        Pruebas unitarias
    └── RateLimitingTests.cs
```

La estructura del cliente sigue la separación entre el núcleo transversal y las funcionalidades.

```
frontend/gestion-activos-ti/src/
├── app/
│   ├── core/
│   │   ├── guards/       Protección de rutas
│   │   ├── interceptors/ Token y manejo de errores
│   │   ├── models/       Interfaces de datos
│   │   └── services/     Consumo de la API
│   ├── features/
│   │   ├── auth/login/
│   │   ├── dashboard/
│   │   ├── activos/      list, form, detail
│   │   ├── responsables/
│   │   ├── historial/
│   │   └── reportes/
│   ├── shared/components/ layout, navbar, diálogos
│   ├── app.routes.ts
│   └── app.config.ts
├── environments/
└── styles.scss
```

Configuración del Entorno

Antes de instalar el sistema deben estar disponibles los componentes que se indican a continuación.

Tabla 98

Requisitos Previos del Entorno

Componente	Versión mínima	Observación
SQL Server	2019	Con SQL Server Management Studio 19 o superior

Componente	Versión mínima	Observación
		para ejecutar los archivos de comando.
SDK de .NET	8.0	Necesario para compilar y ejecutar la interfaz de programación.
Node.js	18	Necesario para compilar el cliente Angular.
Angular CLI	17	Se instala globalmente con el gestor de paquetes de Node.

La configuración del servidor se concentra en el archivo appsettings.json. Las secciones relevantes son las siguientes.

Tabla 99

Parámetros de Configuración del Servidor

Parámetro	Descripción
ConnectionStrings:DefaultConnection	Cadena de conexión a SQL Server. Debe ajustarse el nombre del servidor al del equipo donde se instale la base de datos.
JwtSettings:SecretKey	Clave simétrica con la que se firma el token. Debe tener al menos 32 caracteres y ser distinta en cada ambiente.
JwtSettings:Issuer y Audience	Emisor y destinatario declarados dentro del token.
JwtSettings:ExpirationHours	Vigencia del token en horas. El valor configurado es 8.
FileStorage:BasePath	Carpeta donde se almacenan los documentos adjuntos.
Cors:AllowedOrigins	Direcciones desde las cuales se acepta el consumo de la API. En desarrollo corresponde a la dirección local del cliente Angular.

En el cliente, la dirección de la interfaz de programación se define en los archivos de la carpeta environments. El archivo de desarrollo apunta a la dirección local del servidor, mientras que el archivo de producción debe apuntar a la dirección publicada.

La clave de firma del token se encuentra actualmente en el archivo de configuración en texto plano. Antes de una publicación en producción debe trasladarse a una variable de entorno o al almacén de secretos del sistema operativo, de modo que no viaje en el control de versiones.

Despliegue del Sistema

El procedimiento de instalación desde cero consta de tres etapas, que deben ejecutarse en el orden indicado.

Etapa 1: base de datos

Abra SQL Server Management Studio, conéctese a la instancia de destino y ejecute en orden los tres archivos de comando de la carpeta database. El primero crea la base de datos y elimina cualquier versión anterior con el mismo nombre, por lo que no debe ejecutarse sobre un ambiente con información en producción. Al finalizar, el tercer archivo imprime el listado de objetos creados, que sirve como verificación.

Etapa 2: interfaz de programación

```
cd backend/GestionActivosTI.API
dotnet restore
dotnet run
```

La interfaz queda publicada en la dirección local en el puerto 5000, y su documentación interactiva en la ruta /swagger, disponible únicamente en el ambiente de desarrollo. Para publicarla en un servidor debe generarse la versión compilada y alojarla en el servidor web, ajustando previamente la cadena de conexión y los orígenes permitidos.

```
dotnet publish -c Release -o ./publicacion
```

Etapa 3: cliente web

```
cd frontend/gestion-activos-ti
npm install
npm start
```

El cliente queda disponible en la dirección local en el puerto 4200. Para generar la versión de producción se compila el proyecto y se publica el contenido de la carpeta resultante en el servidor web.

```
ng build --configuration production
```

Verifique la instalación ingresando con las credenciales de prueba definidas en el archivo de comando de la base de datos y comprobando que el tablero muestre los indicadores. Si el tablero aparece vacío, revise en la consola del navegador que las peticiones a la interfaz de programación no estén siendo rechazadas por la política de orígenes cruzados.

Mantenimiento Preventivo y Correctivo

Mantenimiento preventivo

Tabla 100

Rutinas de Mantenimiento Preventivo

Frecuencia	Tarea	Detalle
Diaria	Respaldo completo de la base de datos	Programar un plan de mantenimiento en SQL Server que genere el respaldo fuera del horario laboral y lo copie a un medio distinto del servidor.
Diaria	Respaldo de la carpeta de documentos adjuntos	La carpeta wwwroot/uploads contiene archivos que no residen en la base de datos y deben respaldarse por separado.
Semanal	Revisión del registro de la aplicación	Verificar que no se acumulen excepciones no controladas registradas por el manejador global.
Mensual	Revisión del informe de vulnerabilidades	Actualizar los componentes cuyo soporte haya finalizado y registrar el cambio de versión en el activo correspondiente.
Trimestral	Actualización de dependencias	Revisar las advertencias de seguridad de los paquetes del cliente y del servidor, y aplicar las actualizaciones menores en un ambiente de prueba antes de producción.
Semestral	Rotación de la clave de firma del token	Sustituir la clave simétrica. La rotación invalida las sesiones abiertas, por lo que debe realizarse fuera del horario laboral.
Anual	Depuración de la bitácora	Archivar los registros de HistorialCambios y AuditoriaActivos anteriores al período de retención definido por la organización.

Mantenimiento correctivo

Tabla 101

Diagnóstico de Fallas Frecuentes

Síntoma	Causa probable	Acción correctiva
El cliente muestra «No se pudo conectar con el servidor»	La interfaz de programación no está en ejecución o cambió de puerto.	Verificar que el proceso esté activo y que la dirección configurada en el archivo de entorno del cliente coincida con la del servidor.
Las peticiones se rechazan sin llegar al servidor	La dirección del cliente no está incluida en los orígenes permitidos.	Agregar la dirección en la sección Cors del archivo de configuración y reiniciar la interfaz de programación.
Todas las sesiones expiran de inmediato	La clave de firma del token cambió o difiere entre ambientes.	Confirmar que la clave configurada sea la misma con la que se emitieron los tokens vigentes.
No es posible iniciar sesión con ningún usuario	El resumen de contraseña almacenado no fue generado con BCrypt.	Regenerar el resumen con la misma biblioteca que utiliza el servidor y actualizar la columna PasswordHash.
La carga de documentos falla	La carpeta de destino no existe o el proceso no tiene permiso de escritura.	Verificar la existencia de wwwroot/uploads y otorgar permiso de escritura a la cuenta que ejecuta la aplicación.
Un activo aparece con dos responsables vigentes	Se insertó una asignación directamente en la base de datos.	El disparador tr_ActivoResponsable_UnicoActual corrige la situación de forma automática; verificar el resultado consultando la tabla ActivoResponsable.
Los informes no muestran costos	Los activos no tienen registrado el campo Costo.	Completar el costo desde el formulario de edición del activo.

Anexo Técnico: Puntos de Acceso de la Interfaz de Programación

La tabla siguiente resume los puntos de acceso publicados por la interfaz de programación. Todos requieren el token de sesión, con excepción del inicio de sesión.

Tabla 102

Puntos de Acceso de la Interfaz de Programación

Método	Ruta	Función	Rol requerido
POST	/api/auth/login	Autenticar y emitir el token.	Ninguno
POST	/api/auth/verificar-password	Confirmar la contraseña del usuario en sesión.	Autenticado
GET	/api/activos	Listar activos con filtros y paginación.	Autenticado
POST	/api/activos	Registrar un activo.	Autenticado
GET	/api/activos/{id}	Consultar la ficha completa del activo.	Autenticado
PUT	/api/activos/{id}	Actualizar un activo.	Administrador
PATCH	/api/activos/{id}/estado	Cambiar el estado del activo.	Autenticado
DELETE	/api/activos/{id}	Baja lógica del activo.	Administrador
POST	/api/activos/asignar-responsable	Registrar el cambio de custodia.	Autenticado
POST	/api/activos/{id}/componentes	Agregar un componente tecnológico.	Autenticado
PUT	/api/activos/{id}/ componentes/{cid}	Actualizar un componente.	Autenticado
DELETE	/api/activos/{id}/ componentes/{cid}	Eliminar un componente.	Autenticado
GET	/api/responsables	Listar responsables.	Autenticado
POST	/api/responsables	Registrar un responsable.	Autenticado
PUT	/api/responsables/{id}	Actualizar un responsable.	Administrador
GET	/api/responsables/{id}/activos	Listar los activos a cargo de una persona.	Autenticado
POST	/api/responsables/{id}/ desactivar	Dar de baja con traspaso de activos.	Administrador
POST	/api/responsables/{id}/ reactivar	Reactivar a un responsable.	Administrador
GET	/api/historial/reciente	Consultar la bitácora paginada por categoría.	Autenticado
GET	/api/historial/activo/{id}	Consultar la bitácora de un activo.	Autenticado
POST	/api/documentos/subir	Adjuntar un documento al activo.	Autenticado
DELETE	/api/documentos/{id}	Eliminar un documento adjunto.	Administrador
GET	/api/reportes/resumen	Indicadores del tablero.	Autenticado

Método	Ruta	Función	Rol requerido
GET	/api/reportes/por-vencer	Informe de vencimientos próximos.	Autenticado
GET	/api/reportes/gastos-por-responsable	Informe de gasto por responsable.	Autenticado
GET	/api/reportes/vulnerabilidades	Informe de componentes en riesgo.	Autenticado
GET	/api/reportes/tipos-activo	Catálogo de tipos de activo.	Autenticado